Участник 5: Android, тесты и безопасность

Отчёт о проделанной работе и план до финала

Хакатон Russtech «хак рвб» · агентский ИИ-пайплайн SpecForge · 30–31 октября 2026

~80%	14/16	57/60	83 с
готовность роли без репетиций и видео	шагов e2e на стенде (2 провала — находки backend)	контрактных проверок API на стенде	шлюз Android в Docker без сети, 3 ГБ, не root

Коротко. За ночь собрана вся моя зона конвейера. Каркас Android и Android-агент проверены на настоящей модели Qwen. Образ песочницы собран и проверен на vm5. Маршрут и эталон событий учитывают правила backend. Контрактные тесты и e2e прогнаны на живом стенде. Security-контур привязывает находки к требованиям безопасности. Осталось перенести агента на платформу участника 1 (ждём его push), один раз прогнать CI в GitLab (ждём доступ), проверить агента на чужом ТЗ и провести репетиции.

Документ без секретов: пароли стенда, vm5 и ключ LLM задаются только переменными окружения.

1. Контекст и моя роль

SpecForge — агентский ИИ-пайплайн. Из бизнес-требований он генерирует анализ, архитектуру, backend, web, Android, тесты, безопасность, Docker/CI и документацию. Код пайплайна универсальный: слов предметной области в нём нет. Кейс №1 (платные парковки СПб) служит эталоном, кейс №2 раскроют только на финальной проверке.

Участник	Зона
1	платформа ИИ: исполнитель агентов, ремонтник, оркестратор (тимлид)
2	анализ: требования FR/NFR/SEC, Gherkin, OpenAPI, документация, трассируемость
3	backend (.NET, Kafka, PostGIS), генераторы клиентов, Docker, CI, стенд на vm5
4	Web/UX: спеки экранов, Angular-админка, VLM-ревьюер скриншотов
5 (я)	**Android (Kotlin) + сквозные тесты (контракт, e2e) + безопасность**

Жюри даёт 15 баллов из 100 за «тестирование и безопасность», и это почти целиком моя зона. Важные условия ТЗ: результаты проверок воспроизводимы и сохраняются в GitLab; в отчёте есть тесты, дефекты, уязвимости, критичность и итоговый статус; «подключённый сканер сам по себе не доказательство»; проверки должны подтверждать требования; секреты в репозитории не хранить.

2. Итог по блокам

Блок	Что сделано и проверено	Готово
Mock-маршрут и эталон	Маршрут по Невскому через демо-зоны 514725 и 514660. Генератор эталона учитывает правила backend: гистерезис 2 точки, выезд → закрытие сессии, смена зоны в течение 300 с	**100%**
Каркас Android	Kotlin + Compose, замороженное ядро, эталон экранов кейса №1. 21 тест, 7 снимков. Скриншот-тесты генерируются из UX-спеки без LLM	**100%**
Docker-образ	specforge/android:jdk21-api36 на vm5. Без сети, 3 ГБ, 512 процессов, не root: шлюз за 83 с, образ 1,88 ГБ	**100%**
Android-агент	На Qwen3.8-27B кейс №1 с нуля проходит со 2-й попытки: 26 тестов, 7 снимков, ~62 тыс. токенов, ~6 мин. Не перенесён на платформу участника 1	**~70%**
Контрактные тесты	Строятся из OpenAPI без LLM. На стенде 60 проверок, 57 прошли, находки переданы	**~90%**
e2e демо-сценарий	На стенде 14 из 16 шагов. Оба провала — находки backend (аудит, session_paid). Запускатель для CI в 07_tests/e2e/ проверен	**~95%**
Security-контур	Semgrep, gitleaks, Trivy, ZAP → находки → SEC-требования → LLM-триаж → отчёт. Реальный скан: PASSED. CI ещё не запускался (нет GitLab)	**~90%**
Репетиции и видео	Ждут ТЗ, UX-спеки и GitLab. Эмулятор на ноутбуке недоступен	**0%**

Итого: около **80%** роли без репетиций и видео, около **72%** с ними. Автотестов на мой код — 36, все зелёные.

3. Как шла работа: этапы, проблемы и решения

Этап 3. Первая сборка Android

- Распаковал каркас этапа 3 и попытался собрать. **Проблема:** в Android Studio встроен только JDK 25, а Gradle 8.14.3 на нём не запускается. **Решение:** поставил Temurin JDK 21 и убрал жёсткий *jvmToolchain(17)*. Теперь проект собирается на любом JDK 17+, это важно для раннеров организаторов.
- Тест нашёл реальную ошибку ядра: *Batcher* не отправлял пачку по таймауту, если первая точка пришла в момент 0. Исправил, пересчитал хеши замороженных файлов.
- Привёл тестовые данные к сиду участника 3 (360 ₽/час, «Казанская пл. дом 2»). Результат: **BUILD SUCCESSFUL**, 21 тест, снимки всех состояний.

Демо-зоны от участника 3

- Настоящие зоны оказались не прямоугольниками, а отрезками бордюра с радиусом 20 м. Научил генератор работать с линиями (и позже с MultiLineString). Проложил маршрут через настоящие точки стоянки и перегенерировал *points.json*.

Этап 4. Android-агент

- Спроектировал цикл: план экранов (JSON) → слой данных → экран за экраном → навигация → шлюз → ремонт (до 3 попыток, модели отдаются только упавшие файлы). Запись вне feature/ отклоняется, замороженное ядро откатывается автоматически.
- Дымовой прогон на «реплей-модели» (эталон вместо LLM) с настоящим Gradle прошёл с 1-й попытки.
- **Доступ к Qwen.** С ноутбука LLM 10.10.10.10 не отвечала: внутренний адрес, а трафик шёл через VPN. Решение — SSH-туннель через vm5.
- **Что выяснилось на реальной модели:** на ремонте она отвечает без разметки файла — путь теперь выводится из *package*. Она повторяет одну ошибку — теперь получает файлы, где объявлены идентификаторы из ошибки, и пометку «измени подход». Однажды неправильным оказался сам тест — ремонт разрешает его чинить, но не ослаблять. Сервер отвечает 429 — повторы с паузой и Retry-After.
- Итог: полный прогон кейса №1 с нуля на Qwen — **ПРОЙДЕН** со 2-й попытки.

Согласование с платформой участника 1

- Участник 1 описал свой контракт (*ToolAgent*, ремонт и токены на стороне платформы, шлюз — команды gradlew). Нашёл несостыковку: участник 2 пишет по старой схеме с папками 01_…/07_…, участник 1 — по именам этапов. Подготовил сообщения обоим.
- По совету участника 1 скриншот-тесты теперь **генерируются из UX-спеки без LLM**. Агент пишет только экраны и *<Экран>ScreenSample(state)*. Если нет снимка, ошибка указывает на Kotlin-файл экрана, и ремонт чинит экран, а не тесты. Проверка выполняется до Gradle, поэтому минуты сборки не тратятся. Прогон на Qwen по новой схеме тоже прошёл.

Эмулятор

- На ноутбуке (AMD Ryzen 5 3550H) в BIOS выключена виртуализация SVM, поэтому эмулятор не запускается. Для работы он не нужен: снимки делает Roborazzi. Для видео — включить SVM или снять на реальном телефоне.

Docker-образ песочницы

- Три итерации на vm5: (1) Robolectric клал кэш в /root/.m2 — исправлено через *-Duser.home*; (2) не-root пользователь не мог менять права на служебной папке Gradle — она удаляется после прогрева; (3) chmod отдельным шагом раздувал образ до 8 ГБ — объединил шаги.
- Итог: **шлюз за 83 с** без сети, в 3 ГБ памяти, не от root. 7 снимков, образ 1,88 ГБ.

Этап 7. Security-контур

- Единые находки из SARIF и ZAP; категория определяется по сканеру, CWE или словам; SEC-номер берётся из requirements.json по категории (номера меняются от прогона к прогону). LLM-триаж пачками,

секреты в модель не отправляются, вердикты кэшируются. Повторный скан помечает «исправлено».

- Тест поймал нечестность: отчёт ставил «нарушений нет» требованию входа по JWT только потому, что отработали сканеры. Теперь у каждой категории есть основные проверки (для authn, authz и 429 — контракт, для аудита — e2e), а без них ставится «нужна проверка».
- Реальный скан нашего кода (Semgrep, gitleaks, Trivy): 8 находок, 6 ложных. Вердикты сверил вручную. Нашлась реальная проблема в шаблоне клиента участника 3: Level.BODY пишет токены в лог.
- Попутно исправлено: временный сбой LLM попадал в кэш; Windows-пути Semgrep; моё правило давало ложное срабатывание.

Этап 5. Контрактные тесты

- Проверки строятся из OpenAPI: 401 без токена и с поддельным, 403 по x-roles, чужой id, Idempotency-Key, 429, отсутствие 5xx. На фейковом API с заложенными уязвимостями ловят всё, ложных срабатываний нет.
- На стенде: **60 проверок, 57 прошли**. Провалы я перепроверил вручную, прежде чем отправлять участнику 3.

Этап 6. e2e демо-сценарий

- Эталон пересчитан под правила backend (ответ участника 3): ZoneExited → SessionClosed(ZoneExit) → ZoneChanged.
- Точкам ставится текущее время, иначе повторный прогон уйдёт в дубликаты. Длительность оплаты подстраивается под правила зоны. Перед стартом сценарий сам приводит машину в исходное состояние.
- Первый прогон вскрыл особенности стенда: там ~2000 настоящих зон СПб вокруг маршрута, минимальная оплата 60 мин, таймеры не ускорены. Второй прогон: 18 из 25 (5 провалов — настройки e2e под реальную карту). После правок третий прогон через запускатель CI: **14 из 16**, оба провала — находки backend.

4. Как всё связано

Цепочка: ТЗ → анализ (2) → API и данные (2, 3) → экраны (4) → backend (3) и Android (5) → тесты и безопасность (5) → сборка и деплой (3) → документация (2). Управляет оркестратор (1).

Мой блок	Беру у	Отдаю
Android-агент	требования (2), screen-spec.android.json (4), Kotlin-клиент из codegen (3), платформа (1)	Android-проект, APK, снимки <screenId>--<state>--mobile.png → VLM-ревьюер (4)
Образ песочницы	каркас Android	specforge/android:jdk21-api36 → платформа (1)
Маршрут и эталон	демо-зоны из сида, правила зон (3)	points.json (Android), expected_events.json (e2e)
Контракт	openapi.yaml, стенд, учётки (3)	contract-junit.xml → трассируемость (2); находки → security
e2e	стенд, журнал событий, уведомления (3)	e2e-junit.xml → (2); находки аудита → security
Security	весь код, образы (3), requirements.json → security (2)	security.json, report.md → отчёт (2); блокировка в CI (3)

Ключевые соглашения

- Теги требований: *FR-n*, *AC-FR-n-NN*, *SEC-n* — в имени теста; в Kotlin — *@Req("FR-n")*.
- Находки безопасности сопоставляются с SEC-требованиями по категории (authn, authz, privacy, audit, input_validation, rate_limiting, secrets).
- Шлюз Android: *screens.py generate/check → gradlew assembleDebug testDebugUnitTest recordRoborazziDebug → screens.py verify*. Ошибки в формате *file:line:column: error:*.
- CI: shell-раннер на vm5 (тег vm5), сканеры через docker run, стадии build → test → package → security → deploy → e2e.
- Секреты только в masked-переменных: CONTRACT_PASSWORD_DRIVER, CONTRACT_PASSWORD_ADMIN, LLM_BASE_URL/MODEL/API_KEY.

5. Находки по стенду (переданы участнику 3)

Находка	Требование	Критичность
Нет лимита частоты на POST /auth/tokens: 150 неверных паролей подряд, ни одного 429	SEC-01, SEC-06	**высокая**
В событиях сессии нет пользователя (аудит неполный)	SEC-04	**высокая**
active-session для несуществующей машины отвечает 204, а не 404 (нет проверки владельца); IDOR не проверить — на стенде одна машина	SEC-02	**средняя**
Разный регистр полей в payload событий (Reason/SessionId и sessionId)	контракт API	**средняя**
Уведомление session_paid не создаётся	FR-13	**средняя**
Таймеры сессии не ускорены TIME_SCALE: предупреждение «за 10 мин» не проверить в e2e	FR-9	**вопрос**
Шаблон Kotlin-клиента: Level.BODY пишет токены в logcat (в нашем приложении не активно)	SEC-03	**средняя**

6. Что осталось сделать

Закрыто 31.10: **выполнено** — повторный e2e после правок (через запускатель CI: 14 из 16, оба провала — находки backend); **выполнено** — сценарий положен в *07_tests/e2e/route_e2e.py* и проверен на стенде.

#	Задача	От кого зависит	Оценка
1	Перенести Android-агента на ToolAgent платформы (agents/android_coder/template, промпты в .md)	участник 1: git push main	~1 ч

#	Задача	От кого зависит	Оценка
2	Переименовать пути входов и выходов под таблицу «этап → папка»	участник 1: AGENT_SDK.md	~30 мин
3	Первый прогон CI: security:*, tests:contract, e2e; правки окружения (образы сканеров, база Trivy)	организаторы: GitLab	~1 ч
4	Агент на чужом ТЗ (case3/case4)	участник 4: UX-спека; участник 3: Kotlin-клиент	~1 ч
5	testTag как в вебе (action, field, submit); манифест Material 3	согласовано с участником 4	~1 ч
6	Репетиции на синтетических ТЗ, финальный прогон, видео	вся команда	—
7	По желанию: Schemathesis, тест лимита /positions (нужна тестовая машина)	участник 3	—

7. Как запускать (из корня проекта)

Перед запуском: туннель к стенду и/или к LLM через vm5 (пароль вводится вручную), пароли — в окружении.

```
# стенд → http://localhost:8080
ssh -p 10005 -N -L 8080:127.0.0.1:8080 team5@85.143.113.230
# Qwen → http://127.0.0.1:4000/v1
ssh -p 10005 -N -L 4000:10.10.10.10:4000 team5@85.143.113.230

# Android-агент (LLM_BASE_URL, LLM_MODEL, LLM_API_KEY в окружении); --replay — без LLM
py -m agents.android_agent.run --in reference/case1 --out build/agent_case1
# контрактные тесты (CONTRACT_PASSWORD_DRIVER / _ADMIN в окружении)
py -m tools.contract.run --openapi reference/case1/02_system/openapi.yaml
   --base http://localhost:8080 --out artifacts/07_tests
   --user driver=driver.demo --user admin=admin
# e2e демо-сценарий
py -m tools.e2e.run --base http://localhost:8080 --out artifacts/07_tests
# security-контур
py -m tools.security.run --target . --out artifacts/07_tests --requirements requirements.json
# автотесты моей части
py -m unittest agents.android_agent.test_android_agent tools.security.test_security
   tools.contract.test_contract tools.e2e.test_e2e
```

Передача команде: архив *participant5_android_tests_security.zip* (174 файла, без секретов и мусора сборки). Начинать с *participant-5-integration.md*.

8. Выводы

- Всё, что выводится из входов, делается без LLM: скриншот-тесты, контрактные проверки, эталон событий. Модель пишет только экраны и разбирает находки. Это экономит токены и даёт воспроизводимость.
- Проверки на живом стенде нашли реальные дефекты, которые не видны в моках: лимиты, аудит, контракт событий.
- Отчёт безопасности честный: требование считается доказанным только основной проверкой, а не фактом запуска сканера.
- Главные риски до финала зависят от других: push платформы, доступ к GitLab, UX-спека.